

Ayyoub CHETOUANI
BTS SIO
Lycée Saint Rémi - Site Léonard De Vinci
Année scolaire 2025-2026



Synthèse de mon Stage à l'entreprise Logic@l Conseils

Stage du 5 janvier au 20 février



Responsable professionnel : Jean-Phillipe Fontaine
Responsable scolaire : Sylvain Guillemain
Professeur principal : Nathalie Bâcle

SOMMAIRE

Remerciements

Présentation de l'entreprise Logic@I Conseils

Réalisation Professionnelle

Remerciements

J'adresse mes remerciements à l'entreprise Logic@l Conseils, notamment au PDG Monsieur HAMDANI Malik et l'équipe Logic@l pour leur accueil et leurs conseils tout au long du stage.

J'aimerais aussi adresser mes remerciements à mon maître de stage Monsieur FONTAINE Jean-Philippe pour ses enseignements et sa patience durant la période de stage, mais également à l'équipe de pôle Recherche et Développement avec qui j'ai travaillé et qui ont su me guider tout au long de la réalisation professionnelle.

Présentation de l'entreprise Logic@l Conseils

Qu'est-ce que Logic@l Conseils ?

Logic@l Conseils est une ESN ou entreprise de services numérique, il s'agit d'une entreprise spécialisée dans la prestation informatique pour les entreprises. En d'autres termes son rôle est d'accompagner les entreprises clientes à travers une évolution technologique par le déploiement de consultant directement chez ses clients afin de répondre à leurs besoins spécifiques en matière de technologie.

Les différents services de Logic@l Conseils

Il existe plusieurs services dans cette entreprise, on peut distinguer :

- Le service RH qui gère l'administration, la partie comptabilité et la vie en entreprise comme gérer les congés payés ou répondre aux questions des consultants sur des sujets liés à l'entreprise et son fonctionnement.
- Le service Recrutement qui gère le recrutement des consultants au sein de la boîte, en cherchant des clients sur des plateformes de recrutement ou en faisant des déplacements dans des lieux cibles comme des écoles ou d'autres endroits, à la suite de ceci, il fait passer des entretiens aux candidats pour savoir si ce dernier correspond à ce dont le client a besoin.
- Le service communication qui est chargé d'effectuer la communication en interne et en externe vis à vis de l'entreprise, sous plusieurs formats comme des articles, des posts sur les réseaux sociaux, etc....
- Le service commercial qui est chargé de persuader le client de louer un ou plusieurs consultants pour son besoin

Pour ma part, j'ai travaillé auprès du pôle Recherche et Développement qui est une structure de l'entreprise qui est en charge de développer des technologies internes à l'entreprise

Les clients

Il existe de nombreux clients liés à Logic@l Conseils, on peut notamment citer :

- Auchan Retail
- Adeo
- AG2R LA MONDIALE
- AXA Banque
- Bonduelle
- Boulanger
- Crédit Agricole Consumer Finance
- Decathlon
- EUROTUNNEL HOLDING
- Kiloutou
- Leroy Merlin
- İDKIDS
- KIABI
- ...

Les partenaires

Il existe de nombreux partenaires liés à Logic@l Conseils, on peut notamment citer :

- Stambia
- Informatica
- Semarchy
- Snowflake

Les domaines d'activités

Logic@l Conseils est spécialisée dans 3 domaines d'activités différents, en particulier sur :

- **Software Engineering:** pôle se concentrant sur les langages de programmation et Frameworks d'aujourd'hui et de demain. Les architectes de ce pôle sont principalement orientés vers le back-end avec Python et Java, avec quelques notes de front-end grâce à Javascript.
- **Data Analytics :** pôle se concentrant sur l'analyse, plus précisément sur ces 4 points:

- **Analyse de l'existant** : Définir des métriques pertinentes partagées par l'entreprise. Identifier les sources de données fiables.
 - **Plans et règles de diffusion** : Transmettre les données selon les règles de diffusion. Sécuriser et fiabiliser les données.
 - **Outillage** : Consolider les indicateurs d'un Data Warehouse et les diffuser. Choisir la de Reporting et de gestion de flux de données.
 - **Actions du management** : Aboutir à une qualité du suivi du business accrue. Conforter la prise de décision du client.
-
- **Data Management** : pôle se concentrant sur la gestion et la sauvegarde des données, notamment le patrimoine informatique de l'entreprise via ces méthodes :
- **Data management** : ensemble de processus, d'outils et de techniques visant à exploiter les données de manière efficace. Il assure la sécurité, la cohérence et la qualité des données, ce qui est crucial pour les entreprises de tous les secteurs. Les données sont devenues un atout stratégique majeur, utilisées pour réduire les coûts, optimiser les processus de gestion et améliorer les campagnes marketing.
 - **Data Quality** : essentielle pour le succès à long terme des stratégies d'une entreprise. Elle désigne la capacité d'une entreprise à maintenir ses données précises et utiles au fil du temps. Avec le temps, certaines données peuvent devenir obsolètes ou incorrectes, perdant ainsi leur valeur et leur utilité. Par exemple, dans une base de données contenant des adresses e-mail de clients, certaines adresses peuvent devenir invalides si les clients changent d'adresse email ou cessent de les utiliser. Il est donc crucial de nettoyer régulièrement la base de données pour s'assurer que les informations reflètent la réalité.
 - **Data Governance**: essentielle pour garantir la cohérence, la qualité et la sécurité des données au sein d'une entreprise. Les incohérences dans les données peuvent avoir des conséquences graves sur leur intégration et leur intégrité, affectant ainsi la précision de la Business Intelligence, du reporting d'entreprise et des analyses. Pour éviter ces problèmes, il est crucial d'instaurer une gouvernance des données.
 - **GDPR/RGPD** : Le RGPD ne concerne que les données personnelles et non celles des personnes morales.

Voici les quatre principes fondamentaux que les entreprises doivent respecter selon le RGPD :

- **Consentement** : Les entreprises doivent obtenir un consentement explicite et positif des individus avant de collecter leurs données. Ce consentement peut être retiré à tout moment par l'individu concerné.
- **Transparence** : Les entreprises doivent être transparentes dans leurs opérations de collecte et de traitement des données. Elles doivent fournir des informations claires sur la manière dont les données seront utilisées.
- **Droits des personnes** : Les individus dont les données sont collectées ont certains droits, notamment le droit de portabilité des données, le droit de demander la suppression de leurs données, et le droit d'accès à leurs données.
- **Responsabilités** : Les entreprises sont responsables de la sécurité des données qu'elles collectent. Elles doivent renforcer la sécurité des données et, dans certains cas, désigner un Data Protection Officer (DPO), surtout si elles traitent des données sensibles.
- **Pseudonymisation** : également connue sous le nom d'anonymisation réversible, est une technique utilisée pour modifier certains attributs dans une base de données afin de protéger les informations personnelles. Cette méthode permet de traiter les données de manière qu'elles ne puissent pas être directement liées à une personne sans l'utilisation d'informations supplémentaires. Cela limite la corrélation entre différentes informations nominatives, augmentant ainsi la sécurité des données personnelles.

Cette technique est souvent employée pour sécuriser les informations personnelles dans les entreprises. Par exemple, une entreprise peut souhaiter limiter l'accès aux informations personnelles de ses clients à certains employés. Dans ce cas, la pseudonymisation peut être utilisée pour remplacer certains attributs sensibles par d'autres valeurs, tout en sécurisant les attributs originaux dans un endroit inaccessible à ces employés.

Avantages de la Pseudonymisation

- **Conformité avec le RGPD** : La pseudonymisation est recommandée par le Règlement Général sur la Protection des Données (RGPD). Le RGPD est plus souple avec les données pseudonymisées par rapport aux données personnelles non traitées, ce qui en fait une technique avantageuse pour le traitement des données.

- **Réduction des risques de violation** : En cas de violation de données pseudonymisées, l'impact sur les personnes concernées est moindre par rapport à une violation de données personnelles non protégées. Cela permet à l'entreprise d'éviter des poursuites judiciaires potentielles qui pourraient être engagées par les personnes affectées.
- **Profiling** : technique essentielle pour garantir la qualité des données au sein d'une entreprise. La qualité des données est directement liée à la manière dont elles sont profilées, et une mauvaise qualité peut entraîner des pertes financières importantes pour les entreprises, car les décisions marketing et commerciales sont souvent basées sur ces données. Le profilage permet d'analyser et de traiter les données pour détecter et corriger les problèmes, améliorant ainsi leur qualité et leur utilité pour l'entreprise.

Le profilage des données consiste à examiner minutieusement les données pour déterminer leur légitimité et leur qualité. Les algorithmes utilisés évaluent les données selon divers critères tels que le percentile, la fréquence, le maximum et le minimum, et analysent les métadonnées pour identifier les erreurs récurrentes et coûteuses dans les bases de données. Cette opération peut être intégrée dans les tâches quotidiennes des spécialistes des données ou être sous-traitée à une entreprise de services numériques.

Comment Logic@l Conseils gère son patrimoine informatique ?

Logic@l Conseils gère son patrimoine informatique de plusieurs manières :

- Au niveau matériel, l'entreprise gère le matériel informatique grâce à l'un de ces partenaires : ADVCOM.
Cette entreprise fournit les ordinateurs pour les consultants et les configure en fonction de ce dont l'entreprise a besoin, tout en configurant des adresses professionnelles pour l'utilisation des services tel que Microsoft.
- Au niveau logiciel, la plupart des logiciels sont open source comme Visual Studio Code ou Git ou bien achetés par l'entreprise comme la suite Microsoft, mais parfois dans le cadre de mission pour des entreprises clientes, l'entreprise qui est démarchée peut donner les accès à un logiciel à un ou plusieurs consultants pour qu'ils mènent à bien la mission.
- Au niveau personnel, afin que les employés de n'importe quel service puissent utiliser à bien les logiciels fournis, que ça soit lors de mission ou en général, ils peuvent être amenés à effectuer une formation plus au moins grande pour

maîtriser les fondements du logiciel, exemple pour la suite Microsoft, pour ceux travaillant dans l'administration ou ailleurs.

Comment Logic@l Conseils effectue sa veille technologique ?

Les consultants de Logic@l Conseils effectuent leur veille technologique via plusieurs méthodes :

- Des formations payées par l'entreprise
- Des certifications
- De l'information récoltée par des recherches personnelles
- Des ateliers animés par des consultant de l'entreprise sur un thème précis

Réalisation Professionnelle

Projet : Open Data

Contexte de la mission :

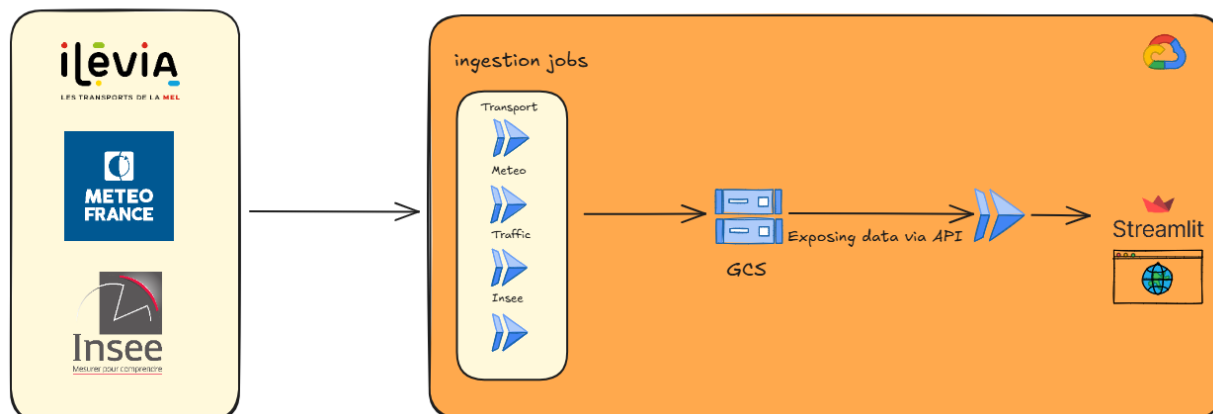
Afin de faciliter l'accès aux clients de l'entreprise à des données libre d'accès venant d'API autour de différents domaines comme la météo ou bien le cours des matières premières, le but est de créer une application utilisant une API regroupant plusieurs données d'autres API qui récoltera les données avec les mises à jour de manière continue et en temps réel.

On m'a donné comme missions au fil de la réalisation de chercher des données liées à plusieurs, de les évaluer au niveau de la pertinence, de les récolter et de les afficher

Besoin :

Schéma montrant le parcours du projet durant les principales étapes :

External Data sources

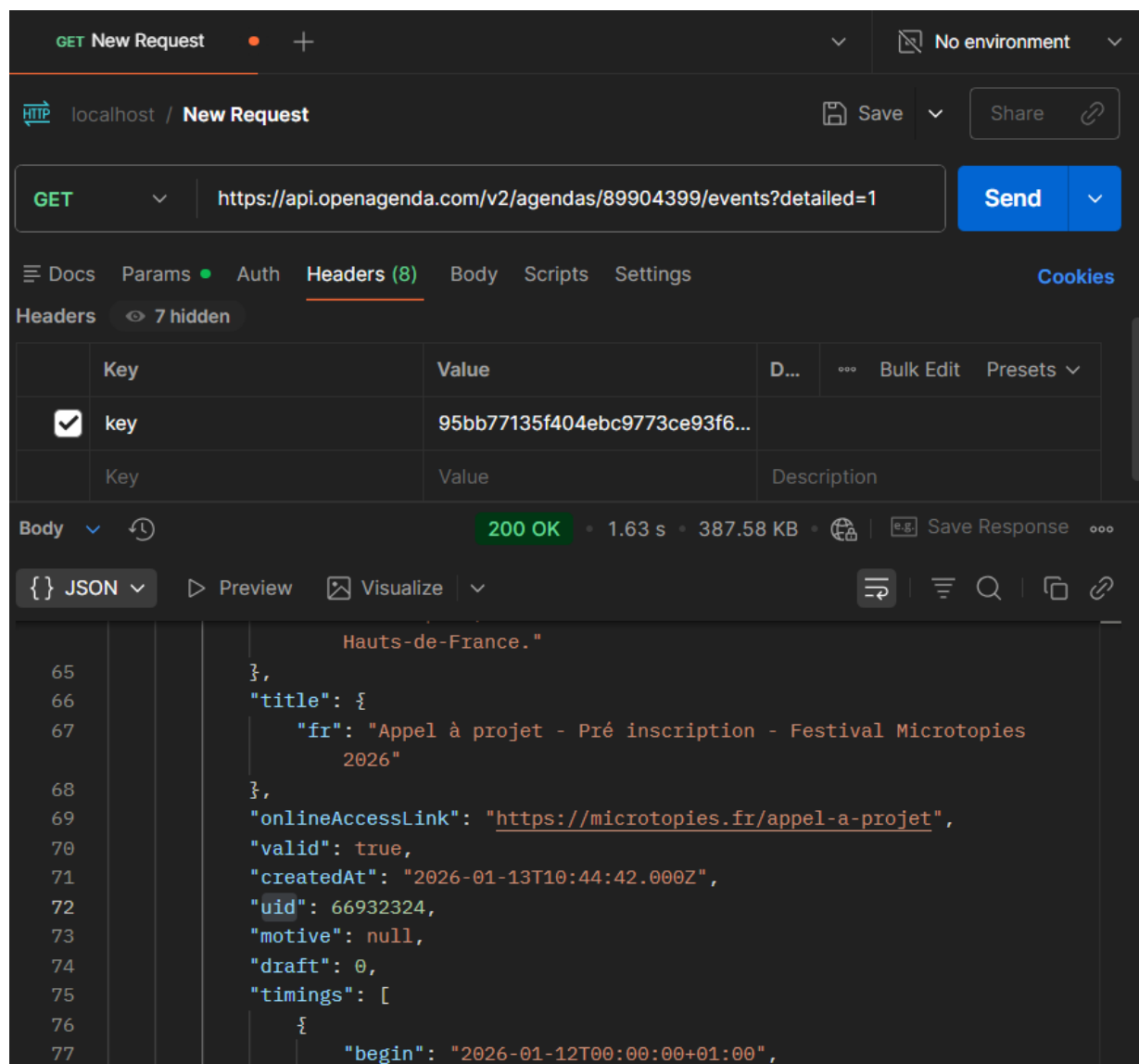


Dans le cadre de la mission j'ai pu découvrir et effectuer toutes les différentes parties, de la recherche des données jusqu'à la conception de l'application et les essais qui vont avec.

Etapas de conception de l'application :

Tout d'abord, il m'a fallu chercher les différentes API correspondantes aux données recherchées comme les données liées à l'inflation, aux matières premières ou encore au cinéma. Pour cela j'ai dû rechercher via plusieurs moyens (recherche google, IA, site du gouvernement data.gouv ...) afin de trouver les bonnes données.

Puis après avoir évalué la pertinence des données vis-à-vis du besoin du projet et au niveau du client, on prend le lien de l'API et on effectue des tests de requête via Postman.



Ici on effectue un test afin de voir ce que propose l'API en termes de données et ce que l'on pourrait exploiter à l'intérieur

Pour donner suite à ces différents tests, on crée un script Python pouvant récupérer les données sous un format JSON avec une structure de répertoires qui permettra aux autres étapes du projet de se fluidifier plus facilement

```
import os
import requests
import pprint
import time
import json

DOSSIER_RACINE = "Banque_France_API"

# Création des dossiers UNE SEULE FOIS
chemin = os.path.join(DOSSIER_RACINE)
os.makedirs(chemin, exist_ok=True)

# Récupération des informations pour plusieurs zones géographiques

url= "https://webstat.banque-france.fr/api/explore/v2.1/catalog/datasets/observations/exports/json/?where=series_key+IN+%28%22ICP.M

response = requests.get(url)

if response.status_code == 200:
    try:
        data_list = response.json() # data_list est une liste
        # if isinstance(data_list, list) and len(data_list) > 0:
        #     data = data_list[0] # prendre le premier dictionnaire
        # else:
        #     data = {}

        for data in data_list:
            # observations_attributes_and_values est une chaîne JSON
            obs_raw = data.get("observations_attributes_and_values", "{}")
            obs_dict = json.loads(obs_raw) # convertir en dictionnaire

            obs_conf = obs_dict.get("OBS_CONF", "")
            obs_status = obs_dict.get("OBS_STATUS", "")

            data_json = {
                "DATASET_ID": data.get("dataset_id", ""),
```

Première partie du code : récupération des données + début du traitement

```

data_json = {
    "DATASET_ID": data.get("dataset_id", ""),
    "CLE_SERIE": data.get("series_key", ""),
    "TITRE": data.get("title_fr", ""),
    "PERIODE": data.get("time_period", ""),
    "PERIODE_DEBUT": data.get("time_period_start", ""),
    "PERIODE_FIN": data.get("time_period_end", ""),
    "VALEUR": data.get("obs_value", ""),
    "STATUT": data.get("obs_status", ""),
    "PUBLICATION_DATE": data.get("updated_at", ""),
    "OBSERVATION": {
        "VALEUR": obs_conf,
        "STATUT": obs_status,
    },
    "BASKETS": data.get("baskets", []),
    "GREATEST_UPDATED_AT": data.get("greatest_updated_at", ""),
    "SERIE_UPLOAD_AT": data.get("series_greatest_updated_at", ""),
    "LAST_THEMES_UPDATED_AT": data.get("last_themes_updated_at", ""),
    "Chemin": data.get("path_fr", "")
}

period_file = str(data.get("time_period", "")) + ".json"
annee = str(data.get("time_period_start", "))[4:]
chemin_period = os.path.join(chemin, annee)
os.makedirs(chemin_period, exist_ok=True)

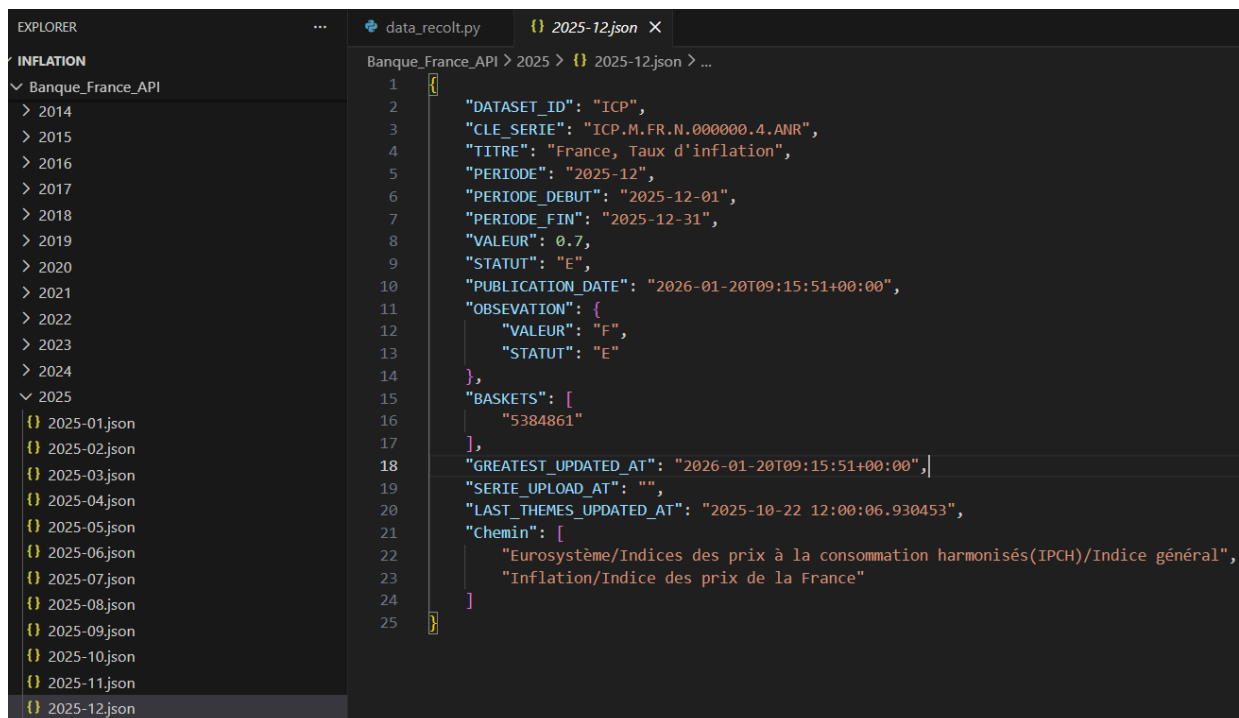
# Sauvegarde JSON
fichier_json = os.path.join(
    chemin_period,
    period_file.replace(' ', '_')
)

with open(fichier_json, "w", encoding="utf-8") as f:
    json.dump(data_json, f, ensure_ascii=False, indent=4)

print(f"Données sauvegardées : {fichier_json}")

```

Dernière partie : Fin du traitement et rangement dans les dossiers en fonction de l'année



Voici le résultat soit l'arborescence des fichiers avec dedans l'un des fichiers JSON retournés et au bon format

Après avoir réussi à obtenir les données sous le bon format et avec aucune erreur ou problème au niveau de la récupération, on adapte le code original du local vers GCP, en ajoutant les dossiers du Bucket et la bibliothèque Python google.cloud

```
import os
import requests
import pprint
import time
import json
from google.cloud import storage

# Configuration
BUCKET_NAME="inflation_bucket_1"
FOLDER="inflation"

# Initialisation client storage
CLIENT=storage.Client()

BUCKET=CLIENT.get_bucket(BUCKET_NAME)
```

Première partie où on configure le bucket et on importe la bibliothèque Google Cloud

```
chemin=f"{FOLDER}/{annee}/{period_file}.json"
blob=BUCKET.blob(chemin)
blob.upload_from_string(data=json.dumps(data_json, ensure_ascii=False), content_type="application/json")
```

Seconde partie où on modifiera le chemin afin de mettre le résultat dans le bucket

Après on pourra mettre le code sur Github, où il y a un dépôt Git contenant le travail actuel

open-data-platform Private Watch 0

feature/test-streamlit 13 Branches 0 Tags Go to file Add file Code

This branch is 80 commits ahead of main. Contribute

StagiaireLogical2 rassemblement des données transport et meteo afin d'avoir chaque mo... 3365a0b · 4 days ago 81 Commits

draft	switch to modular architecture + added new routers	3 months ago
gtfs_static	Premier commit	5 months ago
jobs	ajout des évènements sportifs	5 days ago
meteo	Test swagger asynchrone	2 months ago
routers	ajout de rugby_national_by_period (rugby international)	4 days ago
scripts	Create requirements_local.txt	last month
services	ajout de rugby_national_by_period (rugby international)	4 days ago
streamlit	rassemblement des données transport et meteo afin d'avoir ...	4 days ago
.gitignore	feat(traffic): added Dockerfile, requirement.txt for deployem...	2 months ago
.gitignore.txt	Premier commit	5 months ago
Dockerfile	fix: added asyncio for parallel retrieval of data from gcs	2 months ago
README.md	Update image in README.md	last month
main.py	ajout de rugby_national_by_period (rugby international)	4 days ago
requirements.txt	fix(env): added Dockerfile commands and requirement text	2 months ago

Pour chaque donnée nouvelle que l'on souhaite étudier et tester on doit créer une nouvelle branche basée sur la branche dev qui correspond à la branche de production de l'application.

main default

dev

feature/donnees-cinema

feature/donnees-covid

feature/donnees-festivals

feature/donnees-inflation

feature/donnees-matieres-premieres

feature/donnees-meteo

feature/donnees-sports

feature/test-api-infla-commodity

Les différentes branches

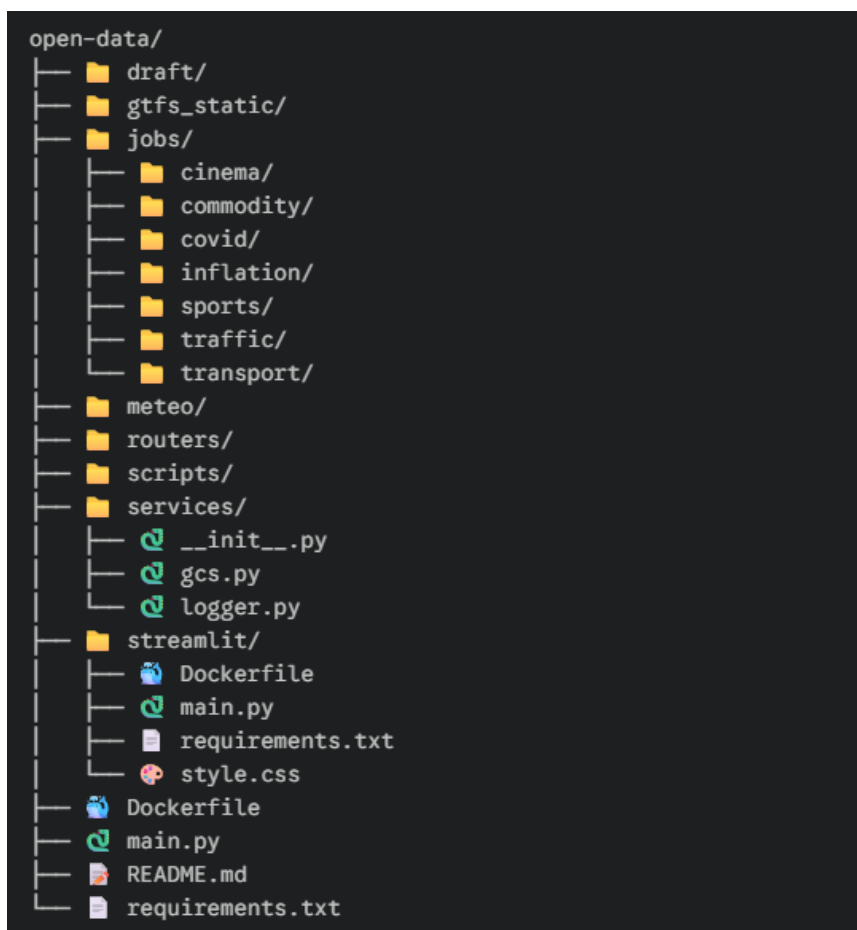
Lorsque l'on est sur Google Cloud Plateform, on a la possibilité de coder directement sur une interface Visual Studio Code, appelée Cloud Run. Arrivé sur Cloud Run, on peut directement mettre notre travail issu de Github, via le changement de branche (git checkout) ou la création de cette dernière (git branch <nom-branche>) afin de le sauvegarder sur Cloud Run.

Maintenant que le dépôt Git est initialisé sur Cloud Run, on peut aborder la partie structure du projet, permettant de savoir quelle fonctionnalité est utilisée pour quelle partie du projet.

On peut distinguer 4 grandes parties dans le Projet OpenData:

- Les jobs : Partie Logique Métier permettant de créer les codes récoltant les données des APIs.
- Les routers : Partie API qui permet de faire des routes et de faire fonctionner l'API Open Data
- Les services : Partie permettant de coder les fonctions liées aux routers et à Streamlit
- Streamlit : Partie Applicative permettant de visualiser le projet

Les technologies utilisées jusqu'à présent furent uniquement Python et quelques bibliothèques comme requests, google.cloud ou encore Json. Mais afin d'utiliser et interagir avec les APIs, on devra utiliser Fast Api qui est un Framework web Python, qui est utilisé pour créer rapidement et simplement des Apis, permettant également de les mettre en production.



On mettra pour chaque type de donnée exploitable, un dossier correspondant dans le dossier `jobs`, dans ce dossier on mettra le code qui récupère les données depuis l'API dans un fichier qui s'appellera `main.py`, il sera accompagné d'un fichier `requirements.txt` qui sera présent pour pouvoir y marquer les dépendances Python comme les bibliothèques et avec un fichier `Dockefile` qui sera là pour pouvoir exécuter le code tout en installant les dépendances dedans.

Sur GCP, afin de faire tourner le code efficacement on a des fonctions plutôt utiles afin de réaliser cela :

- GCP possède une architecture de dossier sous forme de répertoire comme l'explorateur de fichiers ce qui est utile pour mettre nos données qui sont de base dans des dossiers, on stockera nos données dans des Buckets.
- On a la possibilité de créer des images Docker et de les stocker via Artifact Registry, c'est ce qu'on utilisera afin de créer des jobs et des services.
- Les jobs sont créés pour stocker la logique métier soit par exemple les codes utilisés pour récupérer les données.
- Les services sont créés afin de pouvoir visualiser le travail fourni et de le vérifier via des logs.

Afin de créer un job, on créera une image Docker sur Artifact Registry.

Explication de ce qu'est Artifact Registry :

Artifact Registry est un service de Google Cloud qui sert à stocker et gérer des artefacts logiciels, c'est-à-dire tout ce dont tu as besoin pour déployer une application comme des images Docker ou des paquets comme Python, Maven pour Java ou npm pour React.

Artifact Registry

Dépôts

Paramètres

Notes de version

Dépôts

Paramètres

Notes de version

Modifier le dépôt

Supprimer

Instructions de configuration

Filtrer

Saisissez le nom ou la valeur de la propriété

☐	Nom ↑	Format	Type	Zone	Analyse (Désactivé)
☐	cinema-depot	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	cinema-jobs	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	cloud-run-source-deploy	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	cloud-run-source-deploy	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	commodity-depot	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	commodity-jobs	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	covid-depot	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	inflation-depot	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	inflation-jobs	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	meteo-depot	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	open-data-api-repo	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	sport-depot	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	sport-job	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	streamlit-app	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	streamlit-bq	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	streamlit-meteo	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	streamlit-sport	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	test-api-inflation-commodity	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	test-eventarc	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	test-streamlit-cinema	Docker	Standard	europa-west1 (Belgique)	Désactivé
☐	test-streamlit-commodity	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	test-streamlit-covid	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	test-streamlit-inflation	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	traffic-repo	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	transport-api	Docker	Standard	europa-west9 (Paris)	Désactivé
☐	https://console.cloud.google.com/?authuser=1&project=open-data-473312	Docker	Standard	europa-west1 (Belgique)	Désactivé

Interface d'Artifact Registry, ici on peut y voir les dépôts, leur zone et également le moyen d'en créer via la croix +

Artifact Registry / Créer un dépôt

← Créer un dépôt

Nom *

Format
Docker

Mode
☒ Standard
☐ Distant
☐ Virtuel

Type d'emplacement
☒ Région
☐ Multirégional

Région *

Description

Étiquettes
[+ Ajouter une étiquette](#)

Chiffrement
 Par défaut, cette ressource est chiffrée à l'aide d'une clé gérée par Google. Si vous devez gérer votre chiffrement, vous pouvez utiliser une clé gérée par le client à la place. [En savoir plus](#)

☒ Clé de chiffrement gérée par Google
 Clés appartenant à Google

☐ Clé Cloud KMS
 Clés appartenant aux clients

Vous pouvez désormais automatiser la création des clés Cloud KMS à l'aide d'Autokey.

Après avoir créé notre dépôt, on ira créer un Bucket

Cloud Storage

Voici la liste des buckets

Ici on peut voir la manière dont on crée le dépôt, en y déposant un nom et en décidant de la zone du serveur, plus le serveur est proche de nous géographiquement, moins les coûts seront onéreux.

Après l'avoir créé en mettant le nom et la zone où sera stocké le serveur, on retourne sur notre dossier jobs puis dans le dossier voulu en ligne de commande afin de pouvoir ajouter le code dans un Dockerfile, qui sera utilisé afin de faire tourner le code avec cette commande :

```
gcloud builds submit --tag REGION-
docker.pkg.dev/PROJECT_ID/REPO_NAME/IMAGE_NAME:v1 .
```

Dès que le code sera mis dans le job, ce dernier sera exécuté un nombre de fois défini à la manière d'un cron. Par la suite, dans ce job, on créera un swagger permettant d'interroger l'API afin d'obtenir les données voulues, les données voulues seront prises en fonction du moment, comme une date, une période ou même en temps réel. Pour vérifier le résultat du swagger, on déploiera un premier service qui sera d'abord un swagger

Afin de le créer on doit s'attarder sur la partie routers de notre projet :

Les routers sont simplement des routes qui permettent à notre API de pouvoir distinguer les choix possibles par notre utilisateur en fonction de 3 à 4 paramètres possibles d'ordonnement de données :

- By Period : Mesure dans laquelle l'utilisateur souhaite un ordonnancement par période en prenant au minimum 2 dates et au maximum 2 dates et un ou plusieurs éléments supplémentaires comme un type ou un département.
- By Date : Mesure dans laquelle l'utilisateur souhaite un ordonnancement par date en prenant au minimum 1 date et au maximum 1 date et un ou plusieurs éléments supplémentaires comme un type ou un département.
- By Now : Mesure où l'utilisateur souhaite les données en temps réel, il pourra soit directement avoir accès aux données soit, il faudra saisir une variable supplémentaire comme un département ou un type
- By Type : Mesure où l'utilisateur souhaite les données en fonction du type des données.

```
open-data > open-data-platform > routers > commodity_by_type_latest.py > {} APIRouter
1  from fastapi import APIRouter, HTTPException
2  from fastapi.responses import StreamingResponse
3  from services.gcs import get_latest_blob_for_commodity_type
4
5  router = APIRouter(prefix="/commodity", tags=["commodity"])
6
7  @router.get("/{commodity_type}/latest/download")
8  def download_latest(commodity_type: str):
9      blob = get_latest_blob_for_commodity_type(commodity_type)
10
11     if blob is None:
12         raise HTTPException(status_code=404, detail="No JSON found")
13
14     stream = blob.open("rb")
15
16     return StreamingResponse(
17         stream,
18         media_type="application/json",
19         headers={"Content-Disposition": f"attachment; filename={blob.name.split('/')[-1]}"})
20
21
```

Exemple de code de routers : commodity_by_type_latest.py pour les matières premières en fonction de leur type

Dans le dossier services, il y a un dossier appelé gcs.py faisant office de fichier de stockage de toutes les fonctions liées aux routers afin que lors des appels des APIs, les données sont bien envoyées et soient opérationnelles.

```
def get_latest_blob_for_commodity_type(commodity_type: str):
    prefix = f"{FOLDER_COMMODITY}/{commodity_type}.json"

    blobs = client.list_blobs(
        BUCKET_NAME_COMMODITY,
        prefix=prefix
    )

    latest_blob = None

    for blob in blobs:
        if blob.name.endswith(".json"):
            latest_blob = blob
            break # on s'arrête dès qu'on trouve le fichier

    if latest_blob:
        logger.info(f"File found for commodity '{commodity_type}': {latest_blob.name}")
    else:
        logger.info(f"No JSON file found for commodity '{commodity_type}'")

    return latest_blob
```

Code lié à `commodity_by_type_latest.py` afin de pouvoir télécharger sous un unique fichier Json, la somme de plusieurs Json, lors de l'appel

Après avoir complété sa fonction, on doit finir le travail en allant sur le fichier `main.py` qui est à la racine du projet pour importer chaque route du swagger.

```
open-data > open-data-platform > main.py > ...
1  from fastapi import FastAPI
2  from routers.transport_now import router as now_router
3  from routers.transport_by_date import router as date_router
4  from routers.transport_by_period import router as period_router
5  from routers.commodity_by_type_latest import router as commodity_by_type_latest
6  import time
7  from services.logger import logger
8  from routers.traffic_now import router as traffic_now
9  from routers.inflation_date import router as inflation_date
10 from routers.inflation_period import router as inflation_period
11 from routers.cinema_by_date import router as cinema_by_date
12 from routers.cinema_by_period import router as cinema_by_period
13 from routers.cinema_now import router as cinema_now
14 import os
15 from routers.foot_now import router as foot_now
16 from routers.rugby_regional import router as rugby_regional
17 from routers.rugby_national import router as rugby_national
18 from routers.rugby_national_now import router as rugby_national_now
19 from routers.rugby_regional_by_period import router as rugby_regional_by_period
20 from routers.rugby_national_by_period import router as rugby_national_by_period
21
22
23 start = time.time()
24
25 app = FastAPI()
26
27 app.include_router(now_router)
28 app.include_router(date_router)
29 app.include_router(period_router)
30 app.include_router(traffic_now)
31 app.include_router(commodity_by_type_latest)
32 app.include_router(inflation_date)
33 app.include_router(inflation_period)
34 app.include_router(cinema_by_date)
35 app.include_router(cinema_by_period)
36 app.include_router(cinema_now)
37 app.include_router(foot_now)
38 app.include_router(rugby_regional)
39 app.include_router(rugby_regional_by_period)
40 app.include_router(rugby_national)
41 app.include_router(rugby_national_now)
42 app.include_router(rugby_national_by_period)
```

Partie où on importe les différentes routes

```

open-data > open-data-platform > main.py > ...
22
23 start = time.time()
24
25 app = FastAPI()
26
27 app.include_router(now_router)
28 app.include_router(date_router)
29 app.include_router(period_router)
30 app.include_router(traffic_now)
31 app.include_router(commodity_by_type_latest)
32 app.include_router(inflation_date)
33 app.include_router(inflation_period)
34 app.include_router(cinema_by_date)
35 app.include_router(cinema_by_period)
36 app.include_router(cinema_now)
37 app.include_router(foot_now)
38 app.include_router(rugby_regional)
39 app.include_router(rugby_regional_by_period)
40 app.include_router(rugby_national)
41 app.include_router(rugby_national_now)
42 app.include_router(rugby_national_by_period)
43
44
45
46
47 @app.on_event("startup")
48 async def startup_event():
49     logger.info("Starting Transport API...")
50     logger.info(f"Startup time: {time.time() - start:.4f}s")
51
52 @app.on_event("shutdown")
53 async def shutdown_event():
54     logger.info("Shutting down Transport API...")
55
56 @app.get("/")
57 def root():
58     return {"message": "Welcome to the Sport API!"}
59
60 if __name__ == "__main__":
61     import uvicorn
62     port = int(os.environ.get("PORT", 8080))
63     uvicorn.run("main:app", host="0.0.0.0", port=port)
64

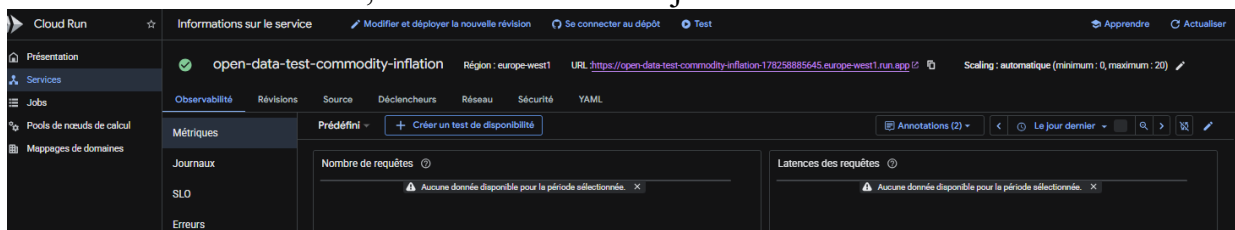
```

Partie où on initialise le swagger

Après un déploiement via la commande :

```
gcloud builds submit --tag REGION-
docker.pkg.dev/PROJECT_ID/REPO_NAME/IMAGE_NAME:v1 .
```

Puis avec l'interface GCP, où il faudra mettre à jour le Dockerfile



Cloud Run

← Déployer la révision sur open-data-test-commodity-inflation (europe-west1)

Chaque modification apportée à la configuration du service entraîne la création d'une révision immuable. Une révision comprend une image de conteneur spécifique ainsi que d'autres paramètres d'environnement.

Conteneurs Réseau Sécurité

Conteneurs

Modifier un conteneur

URL de l'image du conteneur
europe-west1-docker.pkg.dev/open-data-473312/test-api-inflation-commodity/open-data-test Sélectionner

Exemple : un-docker.pkg.dev/locaudrun/containers/hello
Doit écouter les requêtes HTTP sur \$PORT (par défaut : 8080) et ne pas dépendre de l'état local. Comment créer un conteneur ? (2)

Port du conteneur
8080

Les requêtes seront envoyées au conteneur sur ce port. Nous recommandons d'écouter sur le port "\$PORT" plutôt que sur ce numéro de port spécifique.

Paramètres Variables et secrets Volumes

Nom du conteneur : open-data-test-commodity-inflation-1 Edit

Commande du conteneur
Laissez ce champ vide pour utiliser la commande de point d'entrée définie dans l'image de conteneur.

Arguments du conteneur
Arguments transmis à la commande de point d'entrée.

Ressources

Mémoire : 1 GiB Processeur : 1

Mémoire à allouer à chaque instance de ce conteneur Nombre de vCPU (processeurs virtuels) alloués à chaque instance de ce conteneur

☐ GPU Nouveau Associer des GPU à ce conteneur

Contrôles de capacité ⓘ

Vérification du démarrage tcp 8080 every 240s

Délai initial 0s

Délai avant expiration 240s

Seuil d'échec 1

+ Ajouter une vérification d'état

Deployer Annuler

Projet : open-data-473312 Modifier

- Conteneurs de démonstration
- europe-west1-docker.pkg.dev/open-data-473312/cinema-depot
- europe-west1-docker.pkg.dev/open-data-473312/cinema-jobs
- europe-west1-docker.pkg.dev/open-data-473312/cloud-run-source-deploy
- europe-west1-docker.pkg.dev/open-data-473312/inflation-depot
- europe-west1-docker.pkg.dev/open-data-473312/meteo-depot
- europe-west1-docker.pkg.dev/open-data-473312/streamlit-meteo
- europe-west1-docker.pkg.dev/open-data-473312/test-api-inflation-commodity
- europe-west1-docker.pkg.dev/open-data-473312/test-eventarc
- europe-west1-docker.pkg.dev/open-data-473312/test-streamlit-cinema
- europe-west1-docker.pkg.dev/open-data-473312/transport-depot
- europe-west9-docker.pkg.dev/open-data-473312/cloud-run-source-deploy
- europe-west9-docker.pkg.dev/open-data-473312/commodity-depot
- europe-west9-docker.pkg.dev/open-data-473312/commodity-jobs
- europe-west9-docker.pkg.dev/open-data-473312/covid-depot
- europe-west9-docker.pkg.dev/open-data-473312/inflation-jobs
- europe-west9-docker.pkg.dev/open-data-473312/open-data-api-repo
- europe-west9-docker.pkg.dev/open-data-473312/sport-depot
- europe-west9-docker.pkg.dev/open-data-473312/sport-job
- europe-west9-docker.pkg.dev/open-data-473312/streamlit-app
- europe-west9-docker.pkg.dev/open-data-473312/streamlit-bq
- europe-west9-docker.pkg.dev/open-data-473312/streamlit-covid
- europe-west9-docker.pkg.dev/open-data-473312/test-streamlit-commodity
- europe-west9-docker.pkg.dev/open-data-473312/test-streamlit-covid
- europe-west9-docker.pkg.dev/open-data-473312/test-streamlit-inflation
- europe-west9-docker.pkg.dev/open-data-473312/traffic-repo
- europe-west9-docker.pkg.dev/open-data-473312/transport-api
- europe-west9-docker.pkg.dev/open-data-473312/transport-job

Sélectionner Annuler

On a désormais accès à l'interface du Swagger

FastAPI 0.1.0 OAS 3.1

/openapi.json

transport		^
GET	/transport/{transport_type}/now/download	Download Latest
GET	/transport/{transport_type}/{date}/download	Download By Date
GET	/transport/{transport_type}/period-download	Download Period
traffic		^
GET	/traffic/lille/now	Download Latest
commodity		^
GET	/commodity/{commodity_type}/latest/download	Download Latest
default		^
GET	/	Root

On peut ouvrir l'un des onglets et interroger l'API afin d'avoir les données

Artifact Registry / Créer un dépôt

← Créer un dépôt

Nom *

Format
Docker

Mode
☒ Standard
☐ Distant
☐ Virtuel

Type d'emplacement
☒ Région
☐ Multirégional

Région *

Description

Étiquettes
[+ Ajouter une étiquette](#)

Chiffrement
 Par défaut, cette ressource est chiffrée à l'aide d'une clé gérée par Google. Si vous devez gérer votre chiffrement, vous pouvez utiliser une clé gérée par le client à la place. [En savoir plus](#)

☒ Clé de chiffrement gérée par Google
 Clés appartenant à Google

☐ Clé Cloud KMS
 Clés appartenant aux clients

Vous pouvez désormais automatiser la création des clés Cloud KMS à l'aide d'Autokey.

Après avoir créé notre dépôt, on ira créer un Bucket

Cloud Storage

Buckets

Créer

Actualiser

Accéder au chemin

Apprendre

Présentation

Buckets

Surveillance

Paramètres

Storage Intelligence

Ensembles de données Insign...

Configuration

Filtrer

Filtrer les buckets

Afficher les options

	Nom ↑	Création	Type d'emplacement	Zone	Classe de stockage par défaut	Dernière mo
	cinema_bucket	29 janv. 2026, 09:10:00	Multi-region	us	Standard	29 janv. 2026,
	commodity_bucket	8 janv. 2026, 15:26:31	Region	europe-west9	Standard	8 janv. 2026,
	covid_bucket_france	2 févr. 2026, 14:50:08	Multi-region	us	Standard	2 févr. 2026, 1
	gcf-v2-sources-178258885645-europ_	5 févr. 2026, 10:49:25	Region	europe-west9	Standard	5 févr. 2026, 1
	gcf-v2-uploads-178258885645-europ_	5 févr. 2026, 10:49:19	Region	europe-west9	Standard	5 févr. 2026, 1
	inflation_bucket_1	22 janv. 2026, 10:00:24	Region	europe-west9	Standard	22 janv. 2026,
	meteo_opendata	29 sept. 2025, 16:29:09	Multi-region	eu	Standard	29 sept. 2025
	open-data-473312_cloudbuild	28 oct. 2025, 11:30:10	Multi-region	us	Standard	28 oct. 2025,
	run-sources-open-data-473312-europ_	28 oct. 2025, 10:45:23	Region	europe-west1	Standard	28 oct. 2025,
	run-sources-open-data-473312-europ_	26 sept. 2025, 15:18:25	Region	europe-west9	Standard	26 sept. 2025
	sport_bucket_france	9 févr. 2026, 10:59:20	Region	europe-west9	Standard	9 févr. 2026, 1
	traffic_bucket_1	10 déc. 2025, 14:37:42	Region	europe-west9	Standard	10 déc. 2025,
	transport_bucket_1	26 sept. 2025, 15:07:31	Region	europe-west9	Standard	6 févr. 2026, 0

Lignes par page: 30 1 - 13 sur 13

Voici la liste des buckets

Élément	Coût
us (plusieurs régions aux États-Unis)	\$0.026 par Go/mois
Avec réplication par défaut	\$0.020 par Go écrit

Estimer votre coût mensuel

Ici on peut voir la manière dont on crée le dépôt, en y déposant un nom et en décidant de la zone du serveur, plus le serveur est proche de nous géographiquement, moins les coûts seront onéreux.

Après l'avoir créé en mettant le nom et la zone où sera stocké le serveur, on retourne sur notre dossier jobs puis dans le dossier voulu en ligne de commande afin de pouvoir ajouter le code dans un Dockerfile, qui sera utilisé afin de faire tourner le code avec cette commande :

```
gcloud builds submit --tag REGION-
docker.pkg.dev/PROJECT_ID/REPO_NAME/IMAGE_NAME:v1 .
```

Dès que le code sera mis dans le job, ce dernier sera exécuté un nombre de fois défini à la manière d'un cron. Par la suite, dans ce job, on créera un swagger permettant d'interroger l'API afin d'obtenir les données voulues, les données voulues seront prises en fonction du moment, comme une date, une période ou même en temps réel. Pour vérifier le résultat du swagger, on déploiera un premier service qui sera d'abord un swagger

Afin de le créer on doit s'attarder sur la partie routers de notre projet :

Les routers sont simplement des routes qui permettent à notre API de pouvoir distinguer les choix possibles par notre utilisateur en fonction de 3 à 4 paramètres possibles d'ordonnement de données :

- By Period : Mesure dans laquelle l'utilisateur souhaite un ordonnancement par période en prenant au minimum 2 dates et au maximum 2 dates et un ou plusieurs éléments supplémentaires comme un type ou un département.
- By Date : Mesure dans laquelle l'utilisateur souhaite un ordonnancement par date en prenant au minimum 1 date et au maximum 1 date et un ou plusieurs éléments supplémentaires comme un type ou un département.
- By Now : Mesure où l'utilisateur souhaite les données en temps réel, il pourra soit directement avoir accès aux données soit, il faudra saisir une variable supplémentaire comme un département ou un type
- By Type : Mesure où l'utilisateur souhaite les données en fonction du type des données.

```
open-data > open-data-platform > routers > commodity_by_type_latest.py > {} APIRouter
1  from fastapi import APIRouter, HTTPException
2  from fastapi.responses import StreamingResponse
3  from services.gcs import get_latest_blob_for_commodity_type
4
5  router = APIRouter(prefix="/commodity", tags=["commodity"])
6
7  @router.get("/{commodity_type}/latest/download")
8  def download_latest(commodity_type: str):
9      blob = get_latest_blob_for_commodity_type(commodity_type)
10
11     if blob is None:
12         raise HTTPException(status_code=404, detail="No JSON found")
13
14     stream = blob.open("rb")
15
16     return StreamingResponse(
17         stream,
18         media_type="application/json",
19         headers={"Content-Disposition": f"attachment; filename={blob.name.split('/')[-1]}"})
20
21
```

Exemple de code de routers : commodity_by_type_latest.py pour les matières premières en fonction de leur type

Dans le dossier services, il y a un dossier appelé gcs.py faisant office de fichier de stockage de toutes les fonctions liées aux routers afin que lors des appels des APIs, les données sont bien envoyées et soient opérationnelles.

```
def get_latest_blob_for_commodity_type(commodity_type: str):
    prefix = f"{FOLDER_COMMODITY}/{commodity_type}.json"

    blobs = client.list_blobs(
        BUCKET_NAME_COMMODITY,
        prefix=prefix
    )

    latest_blob = None

    for blob in blobs:
        if blob.name.endswith(".json"):
            latest_blob = blob
            break # on s'arrête dès qu'on trouve le fichier

    if latest_blob:
        logger.info(f"File found for commodity '{commodity_type}': {latest_blob.name}")
    else:
        logger.info(f"No JSON file found for commodity '{commodity_type}'")

    return latest_blob
```

Code lié à `commodity_by_type_latest.py` afin de pouvoir télécharger sous un unique fichier Json, la somme de plusieurs Json, lors de l'appel

Après avoir complété sa fonction, on doit finir le travail en allant sur le fichier `main.py` qui est à la racine du projet pour importer chaque route du swagger.

```
open-data > open-data-platform > main.py > ...
1  from fastapi import FastAPI
2  from routers.transport_now import router as now_router
3  from routers.transport_by_date import router as date_router
4  from routers.transport_by_period import router as period_router
5  from routers.commodity_by_type_latest import router as commodity_by_type_latest
6  import time
7  from services.logger import logger
8  from routers.traffic_now import router as traffic_now
9  from routers.inflation_date import router as inflation_date
10 from routers.inflation_period import router as inflation_period
11 from routers.cinema_by_date import router as cinema_by_date
12 from routers.cinema_by_period import router as cinema_by_period
13 from routers.cinema_now import router as cinema_now
14 import os
15 from routers.foot_now import router as foot_now
16 from routers.rugby_regional import router as rugby_regional
17 from routers.rugby_national import router as rugby_national
18 from routers.rugby_national_now import router as rugby_national_now
19 from routers.rugby_regional_by_period import router as rugby_regional_by_period
20 from routers.rugby_national_by_period import router as rugby_national_by_period
21
22
23 start = time.time()
24
25 app = FastAPI()
26
27 app.include_router(now_router)
28 app.include_router(date_router)
29 app.include_router(period_router)
30 app.include_router(traffic_now)
31 app.include_router(commodity_by_type_latest)
32 app.include_router(inflation_date)
33 app.include_router(inflation_period)
34 app.include_router(cinema_by_date)
35 app.include_router(cinema_by_period)
36 app.include_router(cinema_now)
37 app.include_router(foot_now)
38 app.include_router(rugby_regional)
39 app.include_router(rugby_regional_by_period)
40 app.include_router(rugby_national)
41 app.include_router(rugby_national_now)
42 app.include_router(rugby_national_by_period)
```

Partie où on importe les différentes routes

```

open-data > open-data-platform > main.py > ...
22
23 start = time.time()
24
25 app = FastAPI()
26
27 app.include_router(now_router)
28 app.include_router(date_router)
29 app.include_router(period_router)
30 app.include_router(traffic_now)
31 app.include_router(commodity_by_type_latest)
32 app.include_router(inflation_date)
33 app.include_router(inflation_period)
34 app.include_router(cinema_by_date)
35 app.include_router(cinema_by_period)
36 app.include_router(cinema_now)
37 app.include_router(foot_now)
38 app.include_router(rugby_regional)
39 app.include_router(rugby_regional_by_period)
40 app.include_router(rugby_national)
41 app.include_router(rugby_national_now)
42 app.include_router(rugby_national_by_period)
43
44
45
46
47 @app.on_event("startup")
48 async def startup_event():
49     logger.info("Starting Transport API...")
50     logger.info(f"Startup time: {time.time() - start:.4f}s")
51
52 @app.on_event("shutdown")
53 async def shutdown_event():
54     logger.info("Shutting down Transport API...")
55
56 @app.get("/")
57 def root():
58     return {"message": "Welcome to the Sport API!"}
59
60 if __name__ == "__main__":
61     import uvicorn
62     port = int(os.environ.get("PORT", 8080))
63     uvicorn.run("main:app", host="0.0.0.0", port=port)
64

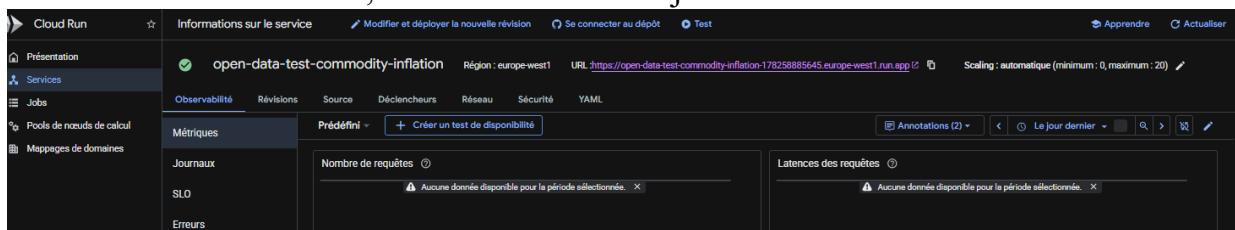
```

Partie où on initialise le swagger

Après un déploiement via la commande :

```
gcloud builds submit --tag REGION-
docker.pkg.dev/PROJECT_ID/REPO_NAME/IMAGE_NAME:v1 .
```

Puis avec l'interface GCP, où il faudra mettre à jour le Dockerfile



On a désormais accès à l'interface du Swagger

On peut ouvrir l'un des onglets et interroger l'API afin d'avoir les données

GET /commodity/{commodity_type}/latest/download Download Latest

Parameters Cancel

Name	Description
commodity_type * required string (path)	argent

Execute

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls: Accept header. Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type: application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }</pre>	No links

Test de l'API Commodity (Matières Premières) afin d'avoir les données sur le cours de l'argent

GET /commodity/{commodity_type}/latest/download Download Latest

Parameters Cancel

Name	Description
commodity_type * required string (path)	argent

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://open-data-api-17825885645.eu-west1.run.app/commodity/argent/latest/download' \
  -H 'accept: application/json'
```

Request URL

```
https://open-data-api-17825885645.eu-west1.run.app/commodity/argent/latest/download
```

Server response

Code	Details
200	Response body Download file Response headers <pre>alt-svc: h3="443"; ma=2592000,h3-29="443"; ma=2592000 content-disposition: attachment; filename=argent.json content-type: application/json date: Mon, 16 Feb 2026 13:27:16 GMT server: Google Frontend</pre>

La réponse de l'API lorsque c'est un succès nous renvoie un fichier téléchargeable
Download file

Autrement une erreur 500 apparaîtra dans le cas où le code possède une erreur et une erreur 404 apparaîtra pour dire qu'il n'y a pas de données récupérables à cet instant

En cliquant sur le lien, on télécharge le dossier en JSON et on obtient le résultat suivant

```
C: > Users > s.quesnoy > Downloads > {} argent (3).json > ...
1  {}
2  "metadata": {
3    "LAST_UPDATE": "2026-01-27",
4    "REF_AREA": "ETR"
5  },
6  "data": [
7    {
8      "Date": "2025-12",
9      "Valeur": 361.4,
10     "Statut": "A",
11     "Qualite": "DEF"
12   },
13   {
14     "Date": "2025-11",
15     "Valeur": 288.7,
16     "Statut": "A",
17     "Qualite": "DEF"
18   },
19   {
20     "Date": "2025-10",
21     "Valeur": 279.7,
22     "Statut": "A",
23     "Qualite": "DEF"
24   },
25   {
26     "Date": "2025-09",
27     "Valeur": 238.6,
28     "Statut": "A",
29     "Qualite": "DEF"
30   },
31   {
32     "Date": "2025-08",
33     "Valeur": 214.9,
34     "Statut": "A",
35     "Qualite": "DEF"
36   },
37   {
```

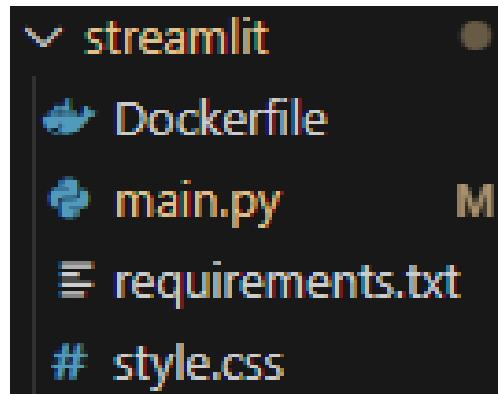
Résultat sur la valeur de l'argent en fonction d'une période d'un mois

Après avoir vérifié que le résultat donné par le swagger est correcte, on peut passer à la partie applicatif et graphique du projet : Streamlit

Streamlit est un framework open-source écrit en Python, conçu pour la création rapide d'applications web interactives dédiées à la Data Science et au Machine Learning. Contrairement aux frameworks web traditionnels (comme Django ou Flask) qui séparent strictement le "Back-end" du "Front-end", Streamlit adopte une approche de développement déclaratif. Il permet aux ingénieurs de transformer des scripts de traitement de données en interfaces utilisateur (UI) sophistiquées en utilisant exclusivement le langage Python, sans nécessiter de compétences en HTML, CSS ou JavaScript.

Mais malgré le CSS n'est pas nécessaire dans le projet, j'ai eu la tâche d'en créer un avec les couleurs de l'entreprise

A la manière des autres parties, le dossier Streamlit du projet, possède un fichier `main.py`, un `requirements.txt` et un `Dockerfile`, mais en plus il peut avoir un fichier `css` qui permet de rajouter un style supplémentaire à la page



Dossier de la partie Streamlit

Le code de streamlit est globalement du Python, mais avec des ajouts qui lui sont propres afin d'avoir une interface propre et simple:

Pour les Textes

- `st.text()` : Texte
- `st.title()` : Titre de l'application
- `st.header()`: En-tête d'une section
- `st.markdown()`: Démarquage d'une section
- `st.subheader()`: Sous-titre d'une section
- `st.caption()`: Légende
- `st.code()`: Code
- `st.latex()`: Expressions mathématiques formatées en LaTeX. ex: `st.latex(r''' a + b = c ''')`

Pour les Medias

- `st.image()`: Image
- `st.audio()`: Audio
- `st.video()`: Vidéo

Pour les Saisies

- `st.number_input()`: Saisie numérique
- `st.text_input()`: Saisie textuelle
- `st.date_input()`: Saisie de date
- `st.time_input()`: Saisie horaire
- `st.text_area()`: Saisie de texte avec moyen d'écrire
- `st.file_uploader()`: Téléchargement de fichiers

Etant donné que l'application en production n'est pas modifiée afin de ne rien altérer,

les captures d'écran suivantes proviennent d'une version créée afin de se rapprocher le plus possible du résultat lors de la mise en production

```
open-data > open-data-platform > streamlit > main.py > {} json
1  # streamlit_app.py
2
3  import streamlit as st
4  import requests
5  import json
6  import pandas as pd
7  import json
8  import datetime
9  import numpy as np
10
11  # =====
12  # Configuration
13  # =====
14
15  API_TRANSPORT = "https://open-data-api-178258885645.europe-west1.run.app"
16  API_METEO = "https://open-data-api-178258885645.europe-west9.run.app"
17  API_COMMODITY= "https://open-data-api-178258885645.europe-west1.run.app"
18  API_INFLATION = "https://inflation-api-178258885645.europe-west1.run.app"
19  API_CINEMAS = "https://cinema-api-178258885645.europe-west1.run.app"
20  API_COVID = "https://covid-depot-178258885645.europe-west9.run.app"
21  API_SPORT = "https://sport-178258885645.europe-west9.run.app"
22  API_EVENTS = "https://events-test-swagger-178258885645.europe-west9.run.app"
23
```

Début du code Streamlit avec l'importation de toutes les APIs

On définit après les variables liées à chaque API

```

# -----
# VARIABLES POUR API TRANSPORTS
# -----
TRANSPORT_TYPES = ["bus", "metro", "tram"]
moments_transport = ["Maintenant", "Par Période", "Par Date"]

# -----
# VARIABLES POUR API COMMODITY
# -----
COMMODITY_CATEGORIES = [
    "argent",
    "petrole_brut",
    "aluminium",
    "cuivre",
    "etain",
    "plomb",
    "zinc",
    "or",
    "platine",
    "cobalt"
]

# -----
# VARIABLES POUR API INFLATION
# -----
moments_inflation = ["Par Période", "Par Date"]

# -----
# VARIABLES POUR API COVID
# -----
moments_covid = ["Par Période", "Par Date"]

# -----
# VARIABLES POUR API METEO
# -----
METEO_CATEGORIES = [
    "temperatures",
    "precipitations",
    "humidite"
]

```

Puis on a la partie configuration des pages de l'application

```

st.title("Open Data API Interface")

# -----
# Sidebar - Navigation
# -----

st.sidebar.title("Navigation")

# Vérification de la connexion des APIs
is_connected_transport, api_response_transport = check_api_connection(API_TRANSPORT)
is_connected_meteo, api_response_meteo = check_api_connection(API_METEO)
is_connected_commodities, api_response_commodities = check_api_connection(API_COMMODITY)
is_connected_inflation, api_response_inflation = check_api_connection(API_INFLATION)
is_connected_cinemas, api_response_cinemas = check_api_connection(API_CINEMAS)
is_connected_covid, api_response_covid = check_api_connection(API_COVID)
is_connected_sport, api_response_sport = check_api_connection(API_SPORT)

if is_connected_transport and is_connected_meteo and is_connected_inflation and is_connected_commodities and is_connected_cinemas and is_connected_covid and is_connected_sport:
    st.sidebar.success("APIs connectées")
else:
    st.sidebar.error("APIs non disponibles")

st.sidebar.markdown("----")

router_choice = st.sidebar.radio(
    "Choisir un service :",
    ["Accueil", "Transport", "Traffic Now", "Meteo", "Matières Premières", "Inflation", "Covid", "Evènements"]
)

# On nettoie les données si on change de page
clear_data_on_page_change(router_choice)

# -----
# Pages
# -----

```

Entre les deux parties, il y a de nombreuses fonctions régissant, le nommage des noms, la conversion du Json au DataFrame afin d'avoir les données sous forme de tableau, afin que l'affichage soit plus agréable à l'utilisateur.

Après avoir vérifier le statut des APIs, on va afficher les différentes routes avec quelques explications sur la nature des données possibles à récupérer

```
st.error("L'API Meteo n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_METEO)

if is_connected_commodities:
    st.success("L'API Commodity est en ligne et opérationnelle.")
    st.json(api_response_commodities)
else:
    st.error("L'API Commodity n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_COMMODITY)

if is_connected_inflation:
    st.success("L'API Inflation est en ligne et opérationnelle.")
    st.json(api_response_inflation)
else:
    st.error("L'API Inflation n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_INFLATION)

if is_connected_cinemas:
    st.success("L'API Cinemas est en ligne et opérationnelle.")
    st.json(api_response_cinemas)
else:
    st.error("L'API Cinemas n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_CINEMAS)

if is_connected_covid:
    st.success("L'API Covid est en ligne et opérationnelle.")
    st.json(api_response_covid)
else:
    st.error("L'API Covid n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_COVID)

if is_connected_sport:
    st.success("L'API Sport est en ligne et opérationnelle.")
    st.json(api_response_sport)
else:
    st.error("L'API Sport n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_SPORT)

if is_connected_events:
    st.success("L'API Events est en ligne et opérationnelle.")
    st.json(api_response_events)
else:
    st.error("L'API Events n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_EVENTS)
```

Statut des APIs

```

router_choice = st.sidebar.radio(
    "Choisir un service :",
    ["Accueil", "Transport", "Traffic Now", "Meteo", "Matières Premières", "Inflation", "Covid", "Evènements"]
)

# On nettoie les données si on change de page
clear_data_on_page_change(router_choice)

# =====
# Pages
# =====

if router_choice == "Accueil":
    st.header("Bienvenue sur l'interface Open Data API")

    st.markdown("""
    Cette interface vous permet d'interagir avec les APIs Open Data pour récupérer
    des données de transport en commun (bus, métro, tram), de météo, du trafic routier,
    le cours des matières premières, les données événementielles (cinéma, sport) et les données liées au Covid 19 .

    ### Services disponibles

    | Service | Description |
    |-----|-----|
    | **Transport** | Récupère les données de transport |
    | **Traffic Now** | Récupère les dernières données de trafic pour Lille |
    | **Meteo** | Récupère les données de météo |
    | **Matières Premières** | Récupère les données des matières premières pour un type spécifique |
    | **Inflation** | Récupère les données de l'inflation |
    | **Covid** | Récupère les données liées au Covid 19 |
    | **Evènements** | Récupère les données événementielles comme les données de cinéma et de sport |

    ### Statut des APIs
    """)

```

Ce qui nous donne en version graphique :

Navigation

APIs connectées

Choisir un service :

- ☒ Accueil
- ☐ Transport
- ☐ Traffic Now
- ☐ Meteo
- ☐ Matières Premières
- ☐ Inflation
- ☐ Covid
- ☐ Evènements

Open Data API Interface v1.0

Open Data API Interface

Bienvenue sur l'interface Open Data API

Cette interface vous permet d'interagir avec les APIs Open Data pour récupérer des données de transport en commun (bus, métro, tram), de météo, du trafic routier, le cours des matières premières, les données événementielles (cinéma, sport) et les données liées au Covid 19 .

Services disponibles

Service	Description
Transport	Récupère les données de transport
Traffic Now	Récupère les dernières données de trafic pour Lille
Meteo	Récupère les données de météo
Matières Premières	Récupère les données des matières premières pour un type spécifique
Inflation	Récupère les données de l'inflation
Covid	Récupère les données liées au Covid 19
Evènements	Récupère les données événementielles comme les données de cinéma et de sport

Statut des APIs

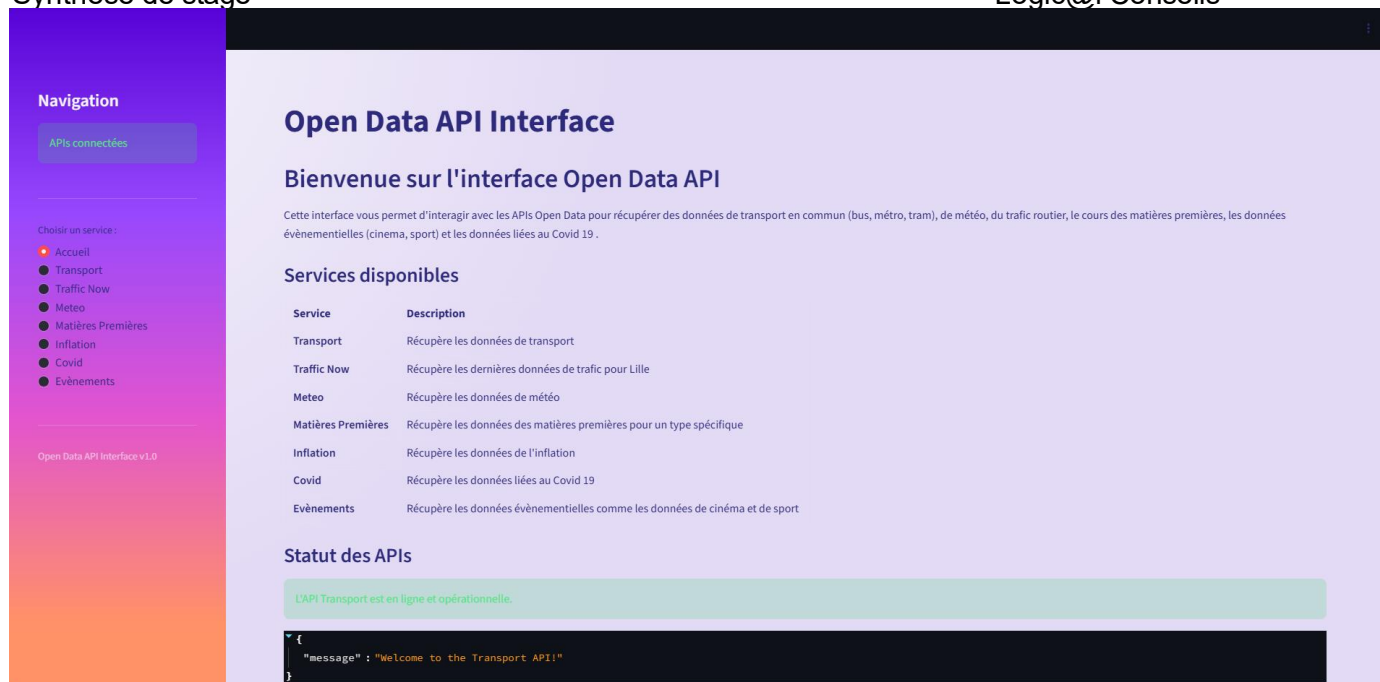
L'API Transport est en ligne et opérationnelle.

```

{
  "message": "Welcome to the Transport API!"
}

```

Avec le CSS ajouté, la page devient ceci :



Pour se diriger vers chaque API c'est via une route que l'on peut y accéder et en fonction de la route, on ira vers le service désiré :

```
elif router_choice == "Evènements":

    evenement = st.selectbox("Type d'Evènement' :", EVENTS)

    if evenement == "Sports":
        st.header("Sports")
        st.markdown("Récupère les données des matchs de différents sports.")

        # Toujours afficher le choix du sport
        sport_type = st.selectbox("Type de Sport :", SPORT_TYPES)

        # Variables par défaut
        endpoint = None
        params = None
        filename = None

        # -----
        # FOOTBALL
        # -----
        if sport_type == "Football":
            sport = "football"

            # Le moment doit TOUJOURS être affiché AVANT toute logique
            moment = st.selectbox("Moment :", moments_football)

            if moment == "Maintenant":
                endpoint = f"/{sport}/latest/download"
                filename = "football_matches_now.json"
```

Après avoir choisi un service, l'utilisateur aura la possibilité de choisir entre les

différentes options qui sont proposées, par exemple pour les Evènements, on en propose de plusieurs sortes :

- Sports
- Cinema
- Evenements Culturels de Métropole Lilloise (Festivals, Atelier, etc ...)

Open Data API Interface

Type d'Evènement :

Sports

Cinema

Evenements Culturels de la Métropole Lilloise

En choisissant le Sport, on se retrouve avec d'autres choix de données comme le Football ou le Rugby :

Open Data API Interface

Type d'Evènement :

Sports

Sports

Récupère les données des matchs de différents sports.

Type de Sport :

Football

Rugby International

Rugby National

En choisissant le Rugby International par exemple, on se retrouve à choisir entre d'autres paramètres afin de savoir la fréquence des données qui doit être renvoyée :

- Maintenant
- Par Période
- Par Date

Open Data API Interface

Type d'Evènement :

Sports

Sports

Récupère les données des matchs de différents sports.

Type de Sport :

Rugby International

Moment :

Maintenant

Par Date

Par Période

Lorsque l'on effectue une recherche on obtient un tableau de ce format :

Type de Sport :
Rugby International

Moment :
Par Date

Année :
2026

Récupérer les données

Données récupérées avec succès ! (4 éléments)

☒ Afficher un aperçu des données ☒ Afficher le bouton de téléchargement

Aperçu des données

Format d'affichage :
☒ Tableau ☐ JSON brut

Nombre de lignes à afficher

	Date du Match	Heure du Match	Localisation	Statut	Pays	Compétition	Saison	Locaux	Challengers	Score des Locaux	Score des Challengers	Mi-temps	Match Complet
0	2026-03-14	20:10	UTC	Not Started	Europe	Six Nations	2026	France	England	None	None	None	None
1	2026-02-05	20:10	UTC	Finished	Europe	Six Nations	2026	France	Ireland	36	14	None	None
2	2026-03-07	14:10	UTC	Not Started	Europe	Six Nations	2026	Scotland	France	None	None	None	None
3	2026-02-15	15:10	UTC	Not Started	Europe	Six Nations	2026	Wales	France	None	None	None	None

Et avec le CSS, cela devient :

Récupère les données des matchs de différents sports.

Type de Sport :
Rugby International

Moment :
Par Date

Année :
2026

Récupérer les données

Données récupérées avec succès ! (4 éléments)

☒ Afficher un aperçu des données ☒ Afficher le bouton de téléchargement

Aperçu des données

Format d'affichage :
☒ Tableau ☐ JSON brut

Nombre de lignes à afficher

	Date du Match	Heure du Match	Localisation	Statut	Pays	Compétition	Saison	Locaux	Challengers	Score des Locaux	Score des Challengers	Mi-temps	Match Complet
0	2026-03-14	20:10	UTC	Not Started	Europe	Six Nations	2026	France	England	None	None	None	None
1	2026-02-05	20:10	UTC	Finished	Europe	Six Nations	2026	France	Ireland	36	14	None	None
2	2026-03-07	14:10	UTC	Not Started	Europe	Six Nations	2026	Scotland	France	None	None	None	None
3	2026-02-15	15:10	UTC	Not Started	Europe	Six Nations	2026	Wales	France	None	None	None	None

Les activités mise en œuvre par rapport au tableau de compétences

Travailler en mode Projet : J'ai pu travailler sur ma réalisation professionnelle en collaboration avec le pôle Recherche et Développement. Grâce à mes collègues j'ai pu avancer et comprendre de mieux en mieux le sujet, également mon maître de stage n'hésitait à faire des points afin de clarifier les consignes et le but dans les tâches que j'avais à faire.

Mettre à disposition des utilisateurs un service informatique : J'ai découvert tout au long du projet Open Data, comment configurer et déployer des services sur GCP

Organiser son développement personnel : J'ai appris à m'informer sur les nouvelles APIs qui peuvent exister notamment celles du gouvernement ou bien celles libre d'accès. Grâce à mes collègues et mon maître de stage, j'ai pu découvrir et chercher des nouvelles librairies Python, élargissant ma culture sur le langage de programmation.

Les difficultés que j'ai rencontrées et comment j'ai pu trouver des solutions ?

Je rencontrais quelques difficultés à cerner les causes des erreurs issues de Python ou bien de GCP, ou encore parfois je bloquais sur un problème à résoudre comme afficher les données dans une dimension précise, par exemple récolter les données en fonction de date et de bien prendre le contenu de chaque date s'il y a un intervalle compris. Il y avait également des difficultés lors des recherches d'API compatible avec notre objectif, la plupart du temps, elles étaient payantes et même dans le cas où elles étaient gratuites, il fallait bien fouiller la documentation et regarder ce que l'API nous renvoyait pour savoir si elle était pertinente ou non

Pour remédier à ces erreurs, il a fallu que je lise attentivement le code ou les erreurs en question, que je cherche dans les logs de GCP, s'il s'agissait des erreurs liées à GCP lors du déploiement. Je devais également me documenter sur des fonctions en Python comme les expressions régulières ou encore les bibliothèques utilisées afin de s'adapter à la nature des données.

Les connaissances que j'ai tiré de ce stage

Bien que je connaisse les langages Python et l'utilisation de GitHub et de Postman j'ai su les approfondir grâce à ce stage.

J'ai surtout découvert les Framework Fast API et Streamlit durant le projet Open Data, j'ai également approfondi mes connaissances sur Google Cloud Platform (GCP), tout en ayant une meilleure maîtrise et une meilleure compréhension de cette technologie

J'ai approfondi également ma manière de travailler en équipe et comment bien se coordonner sur les tâches à se répartir